



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN

[your title here]

Tesis de Licenciatura en Ciencias de la Computación

Rafael Romani

Director: Santiago Cifuentes

Codirector: Santiago Figueira

Buenos Aires, 2026

[YOUR TITLE HERE]

[Acá va el resumen]

Palabras claves: [acá van las palabras clave]

[YOUR TITLE HERE]

[write your abstract here]

Keywords: [write your keywords here]

[acá van los agradecimientos]

CONTENTS

1. Introduction	1
1.1 Uso y poder de los LLMS	1
1.2 Pero qué es un LLM?	1
1.3 Formalizaciones previas y resultados que ignoran los aspectos probabilísticos	1
1.4 Objetivos y contribuciones	1
2. Theoretical Framework	3
2.1 Transformers	3
2.2 Transformer Complexity classes	5
2.3 Circuit Complexity	6
2.4 Known results	6
3. General properties	7
4. Main results	9
5. Conclusions	11

1. INTRODUCTION

- 1.1 Uso y poder de los LLMS
- 1.2 Pero qué es un LLM?
- 1.3 Formalizaciones previas y resultados que ignoran los aspectos probabilísticos
- 1.4 Objetivos y contribuciones

2. THEORETICAL FRAMEWORK

2.1 Transformers

In this work we are going to analyze transformers as language recognizers. A transformer is a machine that works over an alphabet Σ . It has only one tape from where it reads the input and in an autoregressive way writes in it more tokens. There are two distinguished tokens called stopping symbols. Once the transformer writes one of those it halts. We say that a transformer accepts a word if when it halts the stopping symbol printed is the accepting one (q_{accept}). Correspondingly, a transformer rejects a word if the stopping symbol printed is the rejecting one (q_{reject}).

In each step the transformer maps the content of the tape to a probability distribution over the alphabet. Depending on this distribution the transformer prints the next token. The transformer iterates this process until it halts. These iterations are called **chain of thought**.

The process of generating the next token consists of two separated steps. First a probability distribution over the alphabet for the token is computed by a process called distribution generation. After that, it is necessary to decide the specific token by a process called token selection. These two algorithms together is what defines the transformer. During this work we will analyze and compare two different options for token selection.

For a more realistic analysis of the transformers, we don't allow an infinite tape. The transformers studied in this work, for each input size n , will have a tape of size $\text{budget}(n) + n$. It's important to remark that, since in every step of the transformer a symbol is written, the maximum number of steps a transformer can do for an input of length n is $\text{budget}(n)$.

The process of distribution generation is dependent on many parameters. These parameters, in the real models, are the ones calculated during the training. We refer to them as θ . During this work we assume that the parameters force the transformer to halt exactly when the budget runs out. We consider the parameters as part of the definition of a transformer.

We note $\text{TF}(\alpha)$ to the token outputted by one step of the transformer TF when the tape content is $\alpha \in \Sigma^*$. The autoregressive token generation is defined recursively. The token outputted after i steps is $\text{TF}^i(\alpha) := \text{TF}^{i-1}(\alpha \text{TF}(\alpha))$ with base case $\text{TF}^1(\alpha) := \text{TF}(\alpha)$. In other words, the output of the i th step is $\sigma_i = \text{TF}^i(\alpha) = \text{TF}(\alpha \sigma_1 \dots \sigma_{i-1})$ where $\sigma_j \in \Sigma$ for all j .

We note $\text{TF}^*(\alpha)$ for the token written by the transformer when it halts, i.e. at the end of the computation, when the input is α . As was said before, the transformer only halts when it prints a stopping symbol (q_{accept} or q_{reject}). Therefore, $\text{TF}^*(\alpha) \in \{q_{accept}, q_{reject}\}$.

As stated before, the **distribution generation** is a mapping of the content of the tape to a probability distribution over Σ . There are different algorithms for this process in the literature. All the transformers considered in this work use the one presented by [1]. This algorithm consists of four parts: a token embedding layer (TE), a position encoding layer (PE), an output linear layer (OUTPUT), and a stack of L identical decoder layers, where L is also called the depth of the model. Each decoder layer has two sub-layers: a multi-head self-attention layer (ATTN) and a position-wise fully-connected feed-forward network

Write an appendix about this decision. We can always add an extra step or artifact forcing

(FF). Each part and layer has its own trainable parameters. Then, we can define the distribution generation algorithm by its parameters $\theta = \{\theta_{\text{TE}}, \theta_{\text{PE}}, \{\theta_{\text{ATTN}}^l, \theta_{\text{FF}}^l\}_{l=0}^{L-1}, \theta_{\text{OUTPUT}}\}$. Let's define each component.

Token Embedding: It's a mapping from Σ to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\text{TE}} \in \mathbb{R}^{d \times |\Sigma|}$. For all $\sigma \in \Sigma$, we note $\theta_{\text{TE}}(\sigma) \in \mathbb{R}^d$ to the mapping of token σ .

Position Encoding: It's a mapping from the positions of the tape to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\text{PE}} \in \mathbb{R}^{d \times \text{budget}(n) + n}$. For all position of the tape $1 \leq i \leq \text{budget}(n) + n$ we note $\theta_{\text{PE}}(i) \in \mathbb{R}^d$ to the mapping of position i .

Self Attention Layer: It's a mapping from $(\mathbb{R}^d)^a$ to $(\mathbb{R}^d)^a$. Given parameters $\theta_{\text{ATTN}} = (W_Q, W_K, W_V, W_O) \in \mathbb{R}^d \times \mathbb{R}^d \times \mathbb{R}^d \times \mathbb{R}^d$, the layer it's defined by the algorithm 1. This layer is defined only as a single head attention layer because we can simulate 1 multi-head attention layer with any constantly many heads with multiple single-head attention layers. [6]

Algorithm 1 Causal Self-Attention, ATTN

Require: Parameter $\theta_{\text{ATTN}} = (W_Q, W_K, W_V, W_O)$, Input embedding $h = (h_1, \dots, h_a) \in \mathbb{R}^d \times \dots \times \mathbb{R}^d$.

Ensure: Output embedding $h' = (h'_1, \dots, h'_a) := \text{ATTN}_{\theta_{\text{ATTN}}}(h_1, \dots, h_a)$.

- 1: $q_i := W_Q h_i, k_i := W_K h_i, v_i := W_V h_i, \forall i \in [1, \dots, a]$
 - 2: $s_i := \text{softmax}(\langle q_i, k_1 \rangle, \dots, \langle q_i, k_i \rangle) \parallel (0, \dots, 0)$
 - 3: $h'_i := W_O \sum_{j=1}^a (s_i)_j v_j$
-

Feed Forward Network: It's a mapping from $\mathbb{R}^d$ to $\mathbb{R}^d$. Given $\theta_{\text{FF}} = (W_1, b_1, W_2, b_2) \in \mathbb{R}^{d \times d} \times \mathbb{R}^d \times \mathbb{R}^{d \times d} \times \mathbb{R}^d$, it's defined as $\text{FF}_{\theta_{\text{FF}}}(h) := W_2 \text{relu}(W_1 h + b_1) + b_2$, where $\text{relu} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ replaces each position x_i by $\max(0, x_i)$.

Output Layer: It's a mapping from $\mathbb{R}^d$ to a probability distribution over Σ . Given $\theta_{\text{OUTPUT}} \in \mathbb{R}^{|\Sigma| \times d}$, it's defined as $\text{OUTPUT}_{\theta_{\text{OUTPUT}}}(h) := \text{softmax}(\theta_{\text{OUTPUT}} h)$.

All these parts come together for the distribution generator process in the algorithm 2.

Algorithm 2 Distribution generator

Require: Transformer parameter $\theta = (\theta_{\text{TE}}, \theta_{\text{PE}}, \{\theta_{\text{ATTN}}^l, \theta_{\text{FF}}^l\}_{l=0}^{L-1}, \theta_{\text{OUTPUT}})$ and tokens of the tape $\alpha \in \Sigma^*$.

Ensure: Output distribution $p_\theta(\cdot | \alpha)$.

- 1: $h_i^{(0)} \leftarrow \theta_{\text{TE}}(\alpha_i) + \theta_{\text{PE}}(i), \forall 1 \leq i \leq |\alpha|$
 - 2: **for** $l = 0, \dots, L - 1$ **do**
 - 3: $(h_1^{(l+0.5)}, \dots, h_{|\alpha|}^{(l+0.5)}) \leftarrow (h_1^{(l)}, \dots, h_{|\alpha|}^{(l)}) + \text{ATTN}_{\theta_{\text{ATTN}}^l}(h_1^{(l)}, \dots, h_{|\alpha|}^{(l)})$
 - 4: $h_i^{(l+1)} \leftarrow h_i^{(l+0.5)} + \text{FF}_{\theta_{\text{FF}}^l}(h_i^{(l+0.5)}), \forall 1 \leq i \leq |\alpha|$
 - 5: **end for**
 - 6: $p_\theta(\cdot | \alpha) \leftarrow \text{OUTPUT}_{\theta_{\text{OUTPUT}}}(h_{|\alpha|}^{(L)})$
-

The second component of a transformer is the **token selector**. This algorithm consumes the probability distribution computed by the distribution generator and returns a token.

Lets $p : \Sigma \rightarrow [0, 1]$ be the output of the distribution generator. In this work we will study two selectors. The first one always returns the token with the largest probability

(argmax over p , where p is the output of the distribution generator). The other, works non-deterministically. It returns the token $\sigma \in \Sigma$ with probability $p(\sigma)$.

Definition 2.1.1 (Deterministic transformer). We call a transformer deterministic when the token selector is the one taking argmax over the distribution. We say that a family of deterministic transformers $\{\text{TF}_n\}_{n \in \mathbb{N}}$ recognize a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \text{TF}_{|\alpha|}^*(\alpha) = q_{\text{accept}}$$

As mentioned in chapter 1, there is a theoretical gap in the area. *está bien dicho theoretical gap?* A study of non-determinism in transformers is missing.

When TF is a deterministic transformer, $\text{TF}^i(\alpha)$ is an exact symbol for all $\alpha \in \Sigma^*$ and $i \in \mathbb{N}$. In counterpart, when the token selection process of the transformer samples the distribution, it becomes a random variable. Therefore, we introduce the following definition.

Definition 2.1.2 (Probabilistic transformer). We call probabilistic transformers to the transformers that sample the distribution for selecting the next token. We say that a family of probabilistic transformers $\{\text{TF}_n\}_{n \in \mathbb{N}}$ recognize a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \Pr[\text{TF}_{|\alpha|}^*(\alpha) = q_{\text{accept}}] > 2/3$$

2.2 Transformer Complexity classes

Maybe rethink this title?

As discussed in the first chapter, there are different resources to bound in the transformers to limit their expressive power. In this thesis, we will analyze the effect of bounding the number of steps of chain of thought that the transformers are allowed to make before answering. With that in mind, we define the following complexity classes.

Given two functions $T(n)$ and $d(n)$ we define the complexity class $\text{CoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\text{CoT}[T(n), d(n)]$ if and only if there is a deterministic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it.

Definition 2.2.1 ($\text{CoT}[T(n), d(n)]$). A language $\mathcal{L}$ is in $\text{CoT}[T(n), d(n)]$ if and only if there is an integer L and two functions $T'(n) = O(T(n))$, $d'(n) = O(d(n))$, such that for every positive integer n , there is an L -layer deterministic transformer, denoted by TF_n with embedding size $d'(n)$, that can output $\mathcal{L}(\alpha)$ given any input α in Σ^n , using $T'(n)$ steps of chain of thought.

In the same way, we define the classes of languages recognized by probabilistic transformers. Given two functions $T(n)$ and $d(n)$ we define the complexity class $\text{RCoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\text{RCoT}[T(n), d(n)]$ if and only if there is a probabilistic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it.

Definition 2.2.2 ($\text{RCoT}[T(n), d(n)]$). A language $\mathcal{L}$ is in $\text{RCoT}[T(n), d(n)]$ if and only if there is an integer L and two functions $T'(n) = O(T(n))$, $d'(n) = O(d(n))$, such that for every positive integer n , there is an L -layer probabilistic transformer, denoted by TF_n with embedding size $d'(n)$, that can output $\mathcal{L}(\alpha)$ given any input α in Σ^n , using $T'(n)$ steps of chain of thought with probability higher than $2/3$.

Add an image representing the transformer architecture

2.3 Circuit Complexity

2.4 Known results

3. GENERAL PROPERTIES

4. MAIN RESULTS

5. CONCLUSIONS

BIBLIOGRAPHY

- [1] Z. Li, H. Liu, D. Zhou, and T. Ma. Chain of thought empowers transformers to solve inherently serial problems. In The Twelfth International Conference on Learning Representations, 2024.
- [2] A. Madaan and A. Yazdanbakhsh. Text and patterns: For effective chain of thought, it takes two to tango. arXiv preprint arXiv:2209.07686, 2022.
- [3] W. Merrill and A. Sabharwal. A little depth goes a long way: The expressive power of log-depth transformers. arXiv preprint arXiv:2503.03961, 2025.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.
- [5] B. Wang, S. Min, X. Deng, J. Shen, Y. Wu, L. Zettlemoyer, and H. Sun. Towards understanding chain-of-thought prompting: An empirical study of what matters. In A. Rogers, J. Boyd-Graber, and N. Okazaki, editors, Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 2717–2739, Toronto, Canada, July 2023. Association for Computational Linguistics.
- [6] A. Yang, C. Watson, A. Xue, S. Bhattamishra, J. Llarena, W. Merrill, E. D. S. Ferreira, A. Svete, and D. Chiang. The transformer cookbook, 2025.